[MP2] 公開測資依然產生無法結束的 process

9 9

李尚哲 (LI,SHANG-JHE)

All Sections

sleep(1); 只是降低問題出現的機率，並沒有解決問題。經過完整的試驗，這個情況與我的實作無關。

Search entries or author | Unread | Subscribe

Reply

https:/ 楊子慶 (YANG, TZU-CHING) (https://cool.ntu.edu.tw/courses/46296/users/229158)

Mar 27, 2025

同學您好：

如果是指公開測資 18，助教測試自己的實作 1001 次都沒有遇過一次無法結束的問題，可能無法重現同學遇到的錯誤，或者我測不夠多次。

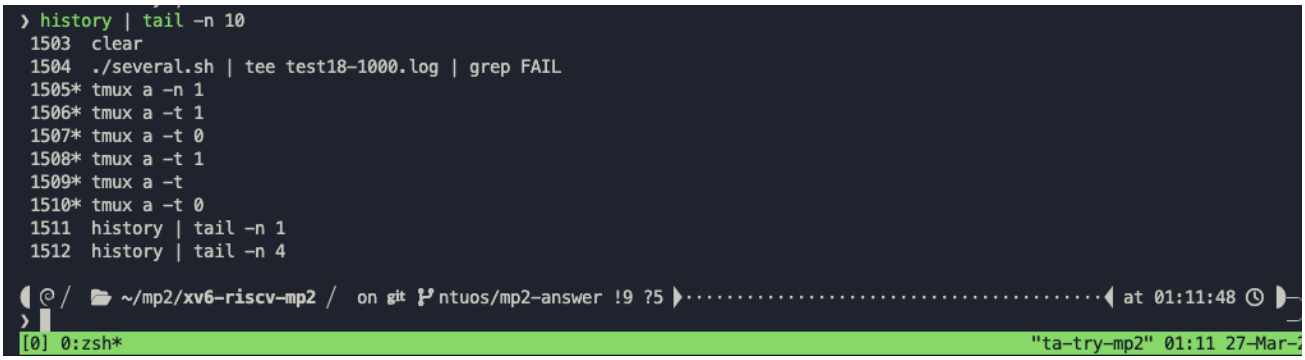
測試過程如下：（grep 找不到 FAIL 所以 exit 1，執行 01:11:30，很抱歉在拉 terminal 時格式有點跑掉了）

several.sh

```
#!/bin/bash

for _ in $(seq 0 1000); do
    ./mp2.sh testcase func 18
done
```

```
~/mp2/xv6-riscv-mp2 / on git ntuos/mp2-answer !9 ?5 0|0|1 ✘ / took 1h 11m 30s / at 01:01:13
❯ ./several.sh | tee test18-1000.log | grep FAIL
```



也許可以請同學於助教時間來討論。如果是其他例子的話，請告訴助教號碼，我再進行確認。

若同學認為已經找到非同學實作部分的問題，可以和助教以及大家分享，讓我們一起學習。

若有更多問題，請不吝提問，感謝。

子慶

Edited by 楊子慶 (YANG, TZU-CHING) (https://cool.ntu.edu.tw/courses/46296/users/229158) on Mar 27 at 1:13am

---

(http

李尚哲 (LI,SHANG-JHE) (https://cool.ntu.edu.tw/courses/46296/users/188748)

Mar 27, 2025

助教時間可以在你的電腦上面測試嗎？可能和測試環境有關？

(http

李尚哲 (LI,SHANG-JHE) (https://cool.ntu.edu.tw/courses/46296/users/188748)

Mar 27, 2025

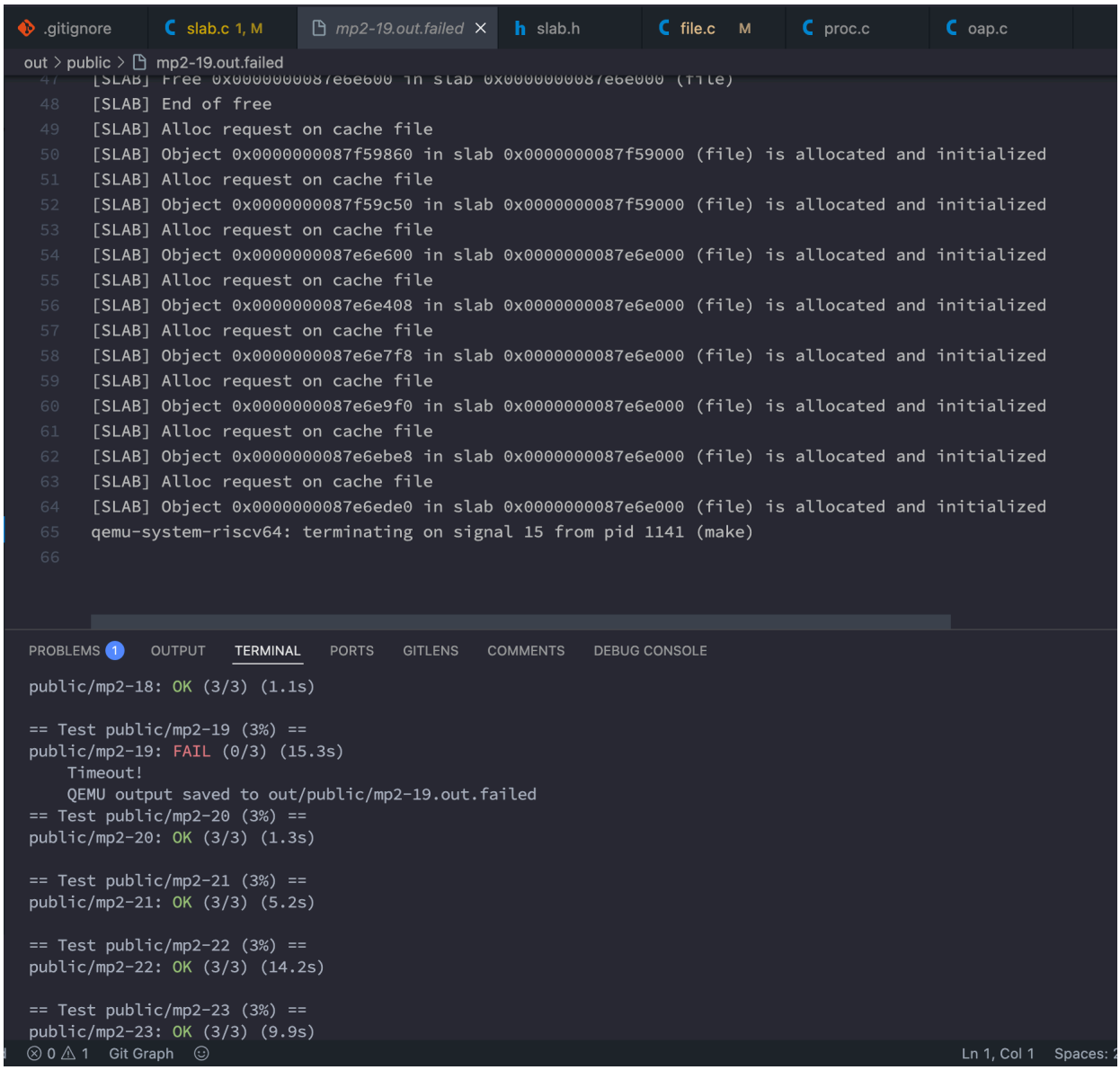
也不是， 18 的問題不是 timeout 我會再想一下，謝謝

(http

李尚哲 (LI,SHANG-JHE) (https://cool.ntu.edu.tw/courses/46296/users/188748)

Mar 27, 2025

之所以會這樣說是因為他已經離開我的 function。而且這樣的情況只出現在 test 裡面有 gah | oak 的情況。


.gitignore
slab.c 1, M
mp2-19.out.failed
slab.h
file.c M
proc.c
oap.c
out > public > mp2-19.out.failed
[SLAB] Free 0x0000000087e6e600 in slab 0x0000000087e6e000 (file)
[SLAB] End of free
[SLAB] Alloc request on cache file
[SLAB] Object 0x0000000087f59860 in slab 0x0000000087f59000 (file) is allocated and initialized
[SLAB] Alloc request on cache file
[SLAB] Object 0x0000000087f59c50 in slab 0x0000000087f59000 (file) is allocated and initialized
[SLAB] Alloc request on cache file
[SLAB] Object 0x0000000087e6e600 in slab 0x0000000087e6e000 (file) is allocated and initialized
[SLAB] Alloc request on cache file
[SLAB] Object 0x0000000087e6e408 in slab 0x0000000087e6e000 (file) is allocated and initialized
[SLAB] Alloc request on cache file
[SLAB] Object 0x0000000087e6e7f8 in slab 0x0000000087e6e000 (file) is allocated and initialized
[SLAB] Alloc request on cache file
[SLAB] Object 0x0000000087e6e9f0 in slab 0x0000000087e6e000 (file) is allocated and initialized
[SLAB] Alloc request on cache file
[SLAB] Object 0x0000000087e6ebe8 in slab 0x0000000087e6e000 (file) is allocated and initialized
[SLAB] Alloc request on cache file
[SLAB] Object 0x0000000087e6ede0 in slab 0x0000000087e6e000 (file) is allocated and initialized
qemu-system-riscv64: terminating on signal 15 from pid 1141 (make)
PROBLEMS 1
OUTPUT
TERMINAL
PORTS
GITLENS
COMMENTS
DEBUG CONSOLE
public/mp2-18: OK (3/3) (1.1s)
== Test public/mp2-19 (3%) ==
public/mp2-19: FAIL (0/3) (15.3s)
Timeout!
QEMU output saved to out/public/mp2-19.out.failed
== Test public/mp2-20 (3%) ==
public/mp2-20: OK (3/3) (1.3s)
== Test public/mp2-21 (3%) ==
public/mp2-21: OK (3/3) (5.2s)
== Test public/mp2-22 (3%) ==
public/mp2-22: OK (3/3) (14.2s)
== Test public/mp2-23 (3%) ==
public/mp2-23: OK (3/3) (9.9s)
0 1
Git Graph
Ln 1, Col 1
Spaces:

在 gah.c 裡面，我們可以看到他會送出他的 pid 之後再 print Ok

```
39    printf("%d", getpid());
40    sleep(1);
41
42    char buf[512];
43    int n;
44
45    while ((n = read(0, buf, sizeof(buf))) > 0)
46    {
47      if (!strcmp(buf, "Ok"))
48      {
49        break;
50      }
51    }
52
53    work(argv[1], atoi(argv[2]));
54
55    sleep(1);
56    printf("Ok");      github-classroom[bot], 6 da
```

在 oak.c 裡面，他會先嘗試讀 pid 然後讀到 Ok 的時候會 kill 剛剛的 pid。

```
6     char buf[512];
7     int n;
8     int pid_to_kill = 0;
9
10    if (!(argc == 2 && !strcmp(argv[1], "end"))) {
11      printf("%d", getpid());
12    }
13
14    while ((n = read(0, buf, sizeof(buf))) > 0)
15    {
16      if (!strcmp(buf, "Ok"))
17      {
18        printf("Ok");
19        kill(pid_to_kill);
20        break;
21      }
22      else      github-classroom[bot], 6 days ago •
23      {
24        pid_to_kill = atoi(buf);
25      }
    }
```

如果你 tracecode 到 piperead (也就是最終他會讀資源的地方)，你會發現他只要有至少一個字元就會 return 。

```
112   acquire(&pi->lock);
113   while(pi->nread == pi->nwrite && pi->writeopen){  //DOC: pipe-empty
114     if(killed(pr)){
115       release(&pi->lock);
116       return -1;
117     }
118     sleep(&pi->nread, &pi->lock); //DOC: piperead-sleep
119   }
120   for(i = 0; i < n; i++){  //DOC: piperead-copy
121     if(pi->nread == pi->nwrite)
122       break;
123     ch = pi->data[pi->nread++ % PIPESIZE];
124     if(copyout(pr->pagetable, addr + i, &ch, 1) == -1)
125       break;
126   }
127   wakeup(&pi->nwrite);  //DOC: piperead-wakeup
128   release(&pi->lock);
```

同時，觀察到 printf 裡面的 putc 是使用 write 一次寫一個字元。

所以可以理解原來的想法是 gah 寫 pid 給 oak 然後等讀到 Ok 之後才送 Ok 給 oak。 oak 接到 pid 之後會記錄下來，然後等到讀到 Ok 之後才 kill <pid of gah>。這裡會出現幾個問題，首先，gah 可能第一次只會讀到 O 隨後他讀到 k，但是因為他都些在同一個位置，所以他會變成 k 也就不等於 Ok ，所以他不會寫 Ok 給 oak。其次，在 oak 裡面，有可能 oak 直接讀到 pid 與 O ，或是讀到 pid 然後只讀到 O ，最後讀到 k 。無論是哪一種，最後的結果都是他不會依照預期的狀況執行。

sleep 一秒可以改善的情況是（oak 直接讀到 pid 與 O），改善的原因是他比較不容易出現，並非不可能。而這並不是所有造成永遠跑不完的原因。

(http

廖昀陽 (YUN-YANG LIAO) (https://cool.ntu.edu.tw/courses/46296/users/181527)

Friday

請問這個問題後來有解決了嗎？
我在 m3 macbook air 上好像還是會遇到卡在 oak 裡面的情況

(http

李尚哲 (LI,SHANG-JHE) (https://cool.ntu.edu.tw/courses/46296/users/188748)

Friday

還沒，我寄信給助教了。

(http

廖昀陽 (YUN-YANG LIAO) (https://cool.ntu.edu.tw/courses/46296/users/181527)

Friday

你有在其他測資上遇到問題嗎？
我在 6, 7, 8 都有遇到過

(http

李怡霈 (LEE I-PEI) (https://cool.ntu.edu.tw/courses/46296/users/220438)

Friday

想問助教有回覆嗎 ~

(http

楊子慶 (YANG, TZU-CHING) (https://cool.ntu.edu.tw/courses/46296/users/229158)

Saturday

同學們好：

根據尚哲同學的建議與來信，助教對 gah、oak、oap 進行優化，設計並實作 read_until_match 函式，同時移除所有 sleep。

這提升了程式碼的效率、重用性、強健性、優雅程度，可見於以下提交：
https://github.com/Shiritai/xv6-ntu-mp2/commit/2b73ae6555e4690a8e97ef4b40c6b92633ce80ac

(https://github.com/Shiritai/xv6-ntu-mp2/commit/2b73ae6555e4690a8e97ef4b40c6b92633ce80ac)
以上程式碼經過 linux、macOS M1、助教以前的低性能 windows 電腦的反覆測試 (各 10 次)，目前沒有遇到錯誤。這表明同學指出的問題確實存在 (利用這次機會助教拿低性能 windows 來測之前的 user program 確實會遇到問題)，並暫時得到解決。若同學們發現其中有任何問題，也請不吝提出，感謝。

子慶

↰ Reply